COL1101

LECTURE -6

AUGUST 4, 2026

Last Class: Object-Oriented Programming



Organize a program around **objects = data + operations** on it, in one unit

Another Example: Set

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        self._items.remove(x)

    def size(self):
        return len(self._items)
```

Another Example: SET

dunder - double underscore

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        self._items.remove(x)

    def size(self):
        return len(self._items)
```

Constructor

Runs once when the object is initialized

Another Example: SET

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        self._items.remove(x)

    def size(self):
        return len(self._items)
```

Data attribute

stores the state

of the object

Another Example: Set

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        self._items.remove(x)

    def size(self):
        return len(self._items)
```

Method:

operations on the data attribute

s = ListSet()

UNDERSCORE CONVENTION -

private , public

_name - convention for "internal, don't touch from outside"

s = ListSet()

✗ s._items.append(x)

-

Underscore Convention In Python

_name - convention for "internal, don't touch from outside"

__name__ (dunder both sides) - reserved for special methods .. Only implement, never invent your own (e.g. __init__, __len__, __str__)

__name - name mangled to _ClassName__name

subclasses won't class it with it by accident.

```
class listset ( ):
    __init__ ( ):
        --.items = [ ]  ⟶  - listset__items
```

Encapsulation Enforces Invariants

Procedural Programming

↳ behave like

```
items = []
def add(items, x):
    if x not in items:
        items.append(x)
```

```
items.append(3); items.append(3)
```

Object-oriented Programming

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)
```

Encapsulation Enforces Invariants

Procedural Programming ✗

```
items = []
def add(items, x):
    if x not in items:
        items.append(x)
```

```
items.append(3); items.append(3)
```

Programmer has to remember to enforce the rules

Object-oriented Programming ✓

```
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)
```

Object guards its own rules

WHAT OBJECTS OFFER : ABSTRACTION

Separate the what from how

```python
class DictSet:
    def __init__(self):
        self._items = {}

    def add(self, x):
        self._items[x] = True

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        del self._items[x]

    def size(self):
        return len(self._items)
```

O(1)

```python
class ListSet:
    def __init__(self):
        self._items = []

    def add(self, x):
        if x not in self._items:
            self._items.append(x)

    def contains(self, x):
        return x in self._items

    def remove(self, x):
        if x not in self._items:
            raise KeyError(x)
        self._items.remove(x)

    def size(self):
        return len(self._items)
```

O(n)

Same what, but completely different how !

TESTING BECOMES EASIER!

```
def test_set(make_set):
    s = make_set()
    assert s.size() == 0
    s.add(3); assert s.contains(3)
    s.add(3); assert s.size() == 1
    s.remove(3); assert not s.contains(3)
    try:
        s.remove(99); assert False
    except KeyError:
        pass

test_set(ListSet)     # passes
test_set(DictSet)     # passes
```

- independent of the implementation
- test individual components of the code

NOTICE: ONE NAME, MANY FORMS

```
def test_set(make_set):
    s = make_set()
    assert s.size() == 0
    s.add(3); assert s.contains(3)
    s.add(3); assert s.size() == 1
    s.remove(3); assert not s.contains(3)
    try:
        s.remove(99); assert False
    except KeyError:
        pass

test_set(ListSet)     # passes
test_set(DictSet)     # passes
```

The same s.add(3) behaves differently at runtime

NOTICE: ONE NAME, MANY FORMS

```
def test_set(make_set):
    s = make_set()
    assert s.size() == 0
    s.add(3); assert s.contains(3)
    s.add(3); assert s.size() == 1
    s.remove(3); assert not s.contains(3)
    try:
        s.remove(99); assert False
    except KeyError:
        pass

test_set(ListSet)     # passes
test_set(DictSet)     # passes
```

The same s.add(3) behaves differently at runtime

dynamic dispatch

Polymorphism: one name / operator / code, working across many types

NOTICE: ONE NAME, MANY FORMS

Q: Any other instances of polymorphism in python?

- `+` on integer : does addition
- `+` on `[]` : concatenates the list

↳ polymorphism → int add (int a, int b)
→ float add (float a, float

Polymorphism: One name / operator / code, working across many types

ABSTRACT DATA TYPE

Model for data types that defines the logical behaviour without telling how it is built in memory

-

One ADT → many implementations!

ABSTRACT DATA TYPE

Model for data types that defines the logical behaviour without telling how it is built in memory

-

Examples

① Set : collection that stores unique elements in no particular order. add, remove, size

② stack : Follows last-in, first-out push, pop, peek

③ Queue : Follows first-in, first-out enqueue, dequeue

ADTs RELY ON SPECIFICATIONS

A good spec has two halves

#1. Operations (typically the following kinds)

- creator: make one from nothing `__init__`
- producer: make one from existing `s.union(s1)`
- observer: read without changing `size()`, `contain()`
- mutator: changes in place `add()`, `remove()`

ADTs RELY ON SPECIFICATIONS

A good spec has two halves

#1. Operations (typically the following kinds)

#2. The behaviour, as precondition & postconditions

```
add(x)      pre:  none
            post: contains(x) is True; size +1 if x was new

remove(x)   pre:  none
            post: contains(x) is False
            raises KeyError if x was absent   <- read twice
```

Where Do Specs Live?

In real code? → the docstring

```
def remove(self, x) -> None:
    """Remove x from the set.

    Effects: x is no longer a member.
    Raises:  KeyError if x was not present.
    """
```

docstring

Enforcing Specification Through Code

How to enforce specs (in Python): Abstract Base Class

```
from abc import ABC, abstractmethod
                  → Abstract Base Class
class Set(ABC):                                    #
    @abstractmethod
    def add(self, x): ...
    @abstractmethod
    def contains(self, x): ...
    @abstractmethod
    def remove(self, x): ...
    @abstractmethod
    def size(self): ...
```

Enforcing Specification Through Code

How to enforce specs (in Python): Abstract Base Class

```
from abc import ABC, abstractmethod

class Set(ABC):                                    #
    @abstractmethod
    def add(self, x): ...
    @abstractmethod
    def contains(self, x): ...
    @abstractmethod
    def remove(self, x): ...
    @abstractmethod
    def size(self): ...
```

class ListSet (Set):
    # implement all 4 abstract methods

Enforcing Specification Through Code

How to enforce specs (in Python): Abstract Base Class

```
from abc import ABC, abstractmethod
                                    # Name1(Name2)
class Set(ABC):
    @abstractmethod
    def add(self, x): ...
    @abstractmethod
    def contains(self, x): ...
    @abstractmethod
    def remove(self, x): ...
    @abstractmethod
    def size(self): ...
```

→ interface

Name1 is inheritep

Name2 class

↓

interface

INTERFACE INHERITENCE

```
class ListSet(Set):
    # implement all 4 methods
```

Enforcing Specification Through Code

How to enforce specs (in Python): Abstract Base Class

```
class BrokenSet(Set):                #
    def add(self, x): ...
    def contains(self, x): ...

BrokenSet() → TypeErr
```

What happens here?

Implementation Inheritence

Inherit to reuse the code

simple

```
class ShoppingCart:                    # Lecture 5's cart + a bulk add
    def __init__(self):    self._items = []
    def add_item(self, item):
        self._items.append(item)
    def add_items(self, items):        # bulk add funnels through add_item
        for item in items: self.add_item(item)

class LimitedCart(ShoppingCart):   # holds at most `max_items`
    def __init__(self, max_items):
        super().__init__(); self._max = max_items
    def add_item(self, item):          # override ONLY add_item
        if len(self._items) >= self._max:
            raise ValueError("cart is full")
        super().add_item(item)
```

def add_item(self, items):

USE IMPLEMENTATION INHERITENCE CAREFULLY!

```
class CountingCart(ShoppingCart):  # analytics:
    def __init__(self):
        super().__init__(); self.num_added = 0
    def add_item(self, item):
        self.num_added += 1
        super().add_item(item)
    def add_items(self, items):
        self.num_added += len(items)    # counts
        super().add_items(items)        # ...base
```

→ 3

c1 = CountingCart()

↳ c1.add_items(["Pen", "Book", "Milk"])

num_added: 6

"Representation exposure"

FIX: HAS-A, NOT IS-A

- default to **has-a** : composition + **delegation**

```
class CountingCart:                                    #
    def __init__(self):
        self._cart = ShoppingCart()
        self.num_added = 0
    def add_item(self, item):
        self.num_added += 1
        self._cart.add_item(item)
    def add_items(self, items):
        self.num_added += len(items)
        self._cart.add_items(items)
```

RECAP

- object : data + operations
- provides abstraction - separate what from how
- "what" written in a spec - defines ADT
- written into code as an interface
- specific data types implement ADT